

Angela Wei

Data 260

Homework 4

Section 0:

Hardware Used: HP Pavilion Windows 11 Pro

Local Model Used:

Commit Hash:

Personal Configuration and Domain

SID4: 1339

PORT_BASE: 8439

PREFIX: s1339

SEED: 1339

VERIFY_SEED: 261339

DOMAIN_ID: 3 – Grocery Supply and Recall Notices

Part 1

App.jsx to connect all jsx pages. All files are gathered in the pages folder.

```
import React, { useEffect, useState } from "react";
import { Routes, Route, useNavigate } from "react-router-dom";
|
import Home from "../pages/Home.jsx";
import CreateUser from "../pages/CreateRecord.jsx";
import UpdateUser from "../pages/UpdateRecord.jsx";
import DeleteUser from "../pages/DeleteRecord.jsx";
```

Get the user's authentication information and start retrieving records if login is successful.

```
//set up for session authentication
const [auth, setAuth] = useState({ loggedIn: false, userId: null });

const [users, setUsers] = useState([]);
const [loading, setLoading] = useState(false);

//get user records if user successfully logs in
useEffect(() => {
  (async () => {
    if (!auth.loggedIn) {
      setUsers([]);
      setLoading(false);
      return;
    }

    try {
      setLoading(true);
      const data = await fetchUsers();
      setUsers(data);
    } catch (e) {
      console.error("fetchUsers failed:", e);
      if (e.message === "Unauthorized") {
        setAuth({ loggedIn: false, userId: null });
      }
    } finally {
      setLoading(false);
    }
  })();
}, [auth.loggedIn]);
```

Create and update user records

```
//create user record info
async function onAdd(newUser) {
  try {
    const created = await createUser(newUser);
    setUsers((prev) => [...prev, created]);
    navigate("/");
  } catch (e) {
    console.error("Create failed:", e);
    if (e.message === "Unauthorized") {
      setAuth({ loggedIn: false, userId: null });
    }
  }
}

//update user record info
async function onUpdate(id, updatedUser) {
  try {
    const updated = await updateUser(id, updatedUser);
    setUsers((prev) => prev.map(u => (u.id === id ? updated : u)));
    navigate("/");
  } catch (e) {
    console.error("Update failed:", e);
    if (e.message === "Unauthorized") {
      setAuth({ loggedIn: false, userId: null });
    }
  }
}
```

Delete the user records:

```
//delete user record info
async function onDelete(id) {
  try {
    await deleteUser(id);
    setUsers((prev) => prev.filter((u) => u.id !== id));
    navigate("/");
  } catch (e) {
    console.error("Delete failed:", e);
    if (e.message === "Unauthorized") {
      setAuth({ loggedIn: false, userId: null });
    }
  }
}
```

Retrieving login information from user in Home.jsx

```
function Home() {
  const [records, setRecords] = useState([]);
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('http://localhost:8080/records', {
      method: 'GET',
      headers: { 'Content-Type': 'application/json' },
      credentials: 'include',
    })
      .then((response) => {
        if (response.status === 401) {
          throw new Error('Login Required');
        }
        if (!response.ok) {
          throw new Error('Unable to get Records');
        }
        return response.json();
      })
      .then((data) => {
        setRecords(data.records);
        setLoading(false);
      })
      .catch((err) => {
        setError(err.message);
        setLoading(false);
      })
  }, []);

  if (loading) return <div>loading your records</div>

  if (error === 'Login Required') return <div>Error: {error}</div>
```

Creating user record information in CreateRecord.jsx

```
function CreateRecord({onAddRecord}) {
  const [name, setName] = useState('');
  const [error, setError] = useState('');
  const [description, setDescription] = useState('');
  const navigate = useNavigate();

  const handleSubmit = (e) => {
    e.preventDefault();

    const newRecordData = {name, description};

    fetch('http://localhost:8080/records', {
      method: 'GET',
      headers: {'Content-Type': 'application/json'},
      credentials: 'include',
      body: JSON.stringify(newRecordData),
    })
    .then((response) => {
      if (response.status === 401){
        throw new Error('Log in Required')
      }
      if (!response.ok) {
        throw new Error('Unable to get record');
      }
      return response.json();
    })
    .then((data) => {
      if (onAddRecord) {
        onAddRecord({id : data.id, ...newRecordData});
      }
      navigate('/');
    })
    .catch((err) => {
      setError(err.message);
    });
  });
};
```

Updating a user record in UpdateRecord.jsx

```
const handleSubmit = (e) => {
  e.preventDefault();

  const updateData = {name, description};

  fetch('http://localhost:8080/records', {
    method: 'GET',
    headers: {'Content-Type': 'application/json'},
    credentials: 'include',
    body: JSON.stringify(updateData),
  })
  .then((response) => {
    if (response.status === 401){
      throw new Error('Log in Required')
    }
    if (!response.ok) {
      throw new Error('Unable to update record');
    }
    return response.json();
  })
  .then((data) => {
    if (onUpdateRecord) {
      onUpdateRecord({id : data.id, ...updateData});
    }
    navigate('/');
  })
  .catch((err) => {
    setError(err.message);
  });
};
```

Deleting a user record in DeleteRecord.jsx

```
const handleDelete = (e) => {
  e.preventDefault();

  const updateData = {name, description};

  fetch('http://localhost:8080/records', {
    method: 'GET',
    headers: {'Content-Type': 'application/json'},
    credentials: 'include',
    body: JSON.stringify(updateData),
  })
  .then((response) => {
    if (response.status === 401){
      throw new Error('Log in Required')
    }
    if (!response.ok) {
      throw new Error('Unable to delete record');
    }
    return response.json();
  })
  .then((data) => {
    if (onDeleteRecord) {
      onDeleteRecord(recordId);
    }
    navigate('/');
  })
  .catch((err) => {
    setError(err.message);
  });
};
```

Part 2

MySQL database called my_db_info is created to store user, user session and records information

```
CREATE DATABASE IF NOT EXISTS my_db_info;
USE my_db_info;

-- table with user credentials
CREATE TABLE IF NOT EXISTS user_info (
  username_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  user_password VARCHAR(255) NOT NULL
);

-- table for server session tokens
-- session id is used as a session token
CREATE TABLE IF NOT EXISTS session_info (
  session_id VARCHAR(255) PRIMARY KEY,
  user_id INT NOT NULL,
  time_created DATETIME DEFAULT CURRENT_TIMESTAMP,
  time_expiration DATETIME NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users(session_id) ON DELETE CASCADE
);

-- table to hold records information
CREATE TABLE IF NOT EXISTS record_info (
  id INT AUTO_INCREMENT PRIMARY KEY,
  primary_field VARCHAR(255) NOT NULL,
  secondary_field TEXT NOT NULL
);
```

Connecting the information to the main FastAPI backend.

```
DATABASE_URL = "mysql+pymysql://your_user:your_password@localhost/myprefix_re1"

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db_connection():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

class User(Base):
    __tablename__ = "user_info"
    username_id = Column(Integer, primary_key=True, index=True)
    name = Column(String(255), nullable=False)
    email = Column(String(255), unique=True, nullable=False)
    user_password = Column(String(255), nullable=False)

class SessionModel(Base):
    __tablename__ = "session_info"
    session_id = Column(String(255), primary_key=True) # Token
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    created_at = Column(DateTime, default=datetime.datetime.utcnow)
    expires_at = Column(DateTime, nullable=False)

class Record(Base):
    __tablename__ = "record_info"
    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    primary_field = Column(String(255), nullable=False)
    secondary_field = Column(Text, nullable=False)
```

Part 3

```
def seed_data():
    #set seed based on assigned seed value
    my_seed = 1339
    random.seed(my_seed)

    db: Session = SessionLocal()

    #clear any data that already exists
    db.query(Comment).delete()
    db.query(Record).delete()
    db.commit()

    #seed the 5000 records
    records = [
        Record(primary_field=f"record: {i}", secondary_field=f"description: {i}")
        for i in range(1, 5001)
    ]
    db.bulk_save_objects(records)
    db.commit()

    #get all existing id records
    all_record_ids = [r.id for r in db.query(Record.id).all()]

    #seed 200 related rows
    comments = []
    for i in range(1, 201):
        rec_id = random.choice(all_record_ids)
        comments.append(
            Comment(record_id=rec_id, comment_text=f"sample: {i}")
        )
    db.bulk_save_objects(comments)
    db.commit()
    db.close()
    print("seeding complete")
```

Create the output table

```
def get_benchmarks():
    page_size = [10, 50, 200]
    total_versions = ["naive", "fixed"]
    n_requests = 30

    def run_benchmark():
        print(f" | Page size | Version | SQL stmts/req | p50 (ms) | p95 (ms) | p99 (ms) |")

        for size in page_size:
            for version in total_versions:
                latencies = []

                # Count expected SQL statements
                sql_stmts = (1 + size) if version == "naive" else 1

                for _ in range(n_requests):
                    start_time = time.perf_counter()
                    res = requests.get(f"{base_url}/{version}?limit={size}")
                    end_time = time.perf_counter()

                    if res.status_code == 200:
                        latencies.append((end_time - start_time) * 1000)

                p50 = np.percentile(latencies, 50)
                p95 = np.percentile(latencies, 95)
                p99 = np.percentile(latencies, 99)

                print(f" | {size} | {version} | {sql_stmts} | {p50:.2f} | {p95:.2f} | {p99:.2f} |")
```

Schema used to create records

```
CREATE TABLE IF NOT EXISTS comment_info (
    id INT AUTO_INCREMENT PRIMARY KEY,
    record_id INT NOT NULL,
    comment_text VARCHAR(255) NOT NULL,
    FOREIGN KEY (record_id) REFERENCES records(id) ON DELETE CASCADE
);
```

Github:

<https://github.com/Aywei2001/data260--SID4->